```
var keyPassword = System.getProperty("keystore.password",
"changeit").toCharArray();
var keyStore = KeyStore.getInstance("PKCS12");
keyStore.load(new FileInputStream("keystore.p12"),
keyPassword);
var macKey = keyStore.getKey("hmac-key", keyPassword);
var encKey = keyStore.getKey("aes-key", keyPassword);
var capController = new CapabilityController(
new MacaroonTokenStore(new JsonTokenStore(), macKey));
```

As currently written, the macaroon token store works exactly like the existing HMAC token store. In the next sections you'll implement support for caveats to take full advantage of the new token format.

9.3.1 First-party caveats

The simplest caveats are first-party caveats, which can be verified by the API purely based on the API request and the current environment. These caveats are represented as strings and there is no standard format. The only commonly implemented first-party caveat is to set an expiry time for the macaroon using the syntax:

```
time < 2019-10-12T12:00:00Z
```

You can think of this caveat as being like the expiry (exp) claim in a JWT (chapter 6). In fact, most of the standard claims in a JWT are caveats on use rather than claims. Listing 9.16 shows how to add an expiry caveat to our macaroons. You should only add this caveat if the token